

P3973R1

bit_cast_as: Element type reinterpretation for std::simd

Published Proposal, 2026-05-12

Author:

[Daniel Towner](#) (Intel Corporation)

Audience:

LEWG, SG1

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose `std::bit_cast_as()`, a facility for reinterpreting `std::simd` objects at different element granularities. This enables safe and efficient operations such as converting packed bytes to shorts, or accessing the underlying bit patterns of float vectors, with compile-time size verification and automatic element count inference. The design suggests a broader pattern for homogeneous array-like types such as `std::array`, and possibly for contiguous views such as `std::span`, though those extensions raise separate design questions and are presented only as future directions. This paper was originally part of [P3445R0], which applied to `simd` only, and is now split out as a more focused proposal. This paper relies on [P3983R0] for the array-like layout guarantees that make the implementation portable and safe.

Table of Contents

- 1 Introduction
- 2 Revision History
- 3 Motivation

- 3.1 The SIMD Element Reinterpretation Problem
 - 3.1.1 Signal Processing - Packed Sample Conversion
 - 3.1.2 Bit Manipulation - Type Punning
 - 3.1.3 Data Packing - Bytes to Words
- 3.2 Intrinsics Already Support This Pattern
- 3.3 Why `std::bit_cast` Doesn't Solve This
- 3.4 What We Want

4 Scope and Design Boundary

5 Proposed Solution

- 5.1 Usage Examples
- 5.2 Relationship to Existing Facilities

6 Design Decisions

- 6.1 Element Count Inference
- 6.2 Size Mismatch Handling
- 6.3 Abi Computation Strategy
- 6.4 Implementation Mechanism
- 6.5 Naming

7 Implementation Experience

- 7.1 Conceptual Implementation

8 Future Directions

- 8.1 Possible Extension to `std::array`
- 8.2 Possible Extension to `std::span`
- 8.3 Non-Targets

9 Design Alternatives Considered

- 9.1 Alternative: Make this a member function
- 9.2 Alternative: Use explicit count parameter
- 9.3 Alternative: Generalize immediately to `array` and `span`

10 Wording

- 10.1 Header `<simd>` synopsis additions
- 10.2 `bit_cast_as` for `simd` [`simd.bit_cast_as`]

11 Acknowledgements

§ 1. Introduction

SIMD programming frequently requires reinterpreting vector data at different element granularities—converting packed bytes to shorts, accessing the bit representation of floats, or regrouping data for different operations. While platform intrinsics have long supported this pattern naturally, with `std::simd` programmers must use `std::bit_cast` with fully-specified target types, manually computing element counts and constructing appropriate ABIs.

This proposal introduces `std::bit_cast_as<T>()`, a facility that brings `std::simd` to parity with platform intrinsics by automatically inferring element counts when reinterpreting SIMD vectors.

Instead of writing:

```
// Verbose: must specify element count and worry about ABI selection
auto shorts = std::bit_cast<vec<uint16_t, 8>>(bytes);
```

Programmers can write:

```
// Clear: element count inferred automatically
auto shorts = std::bit_cast_as<uint16_t>(bytes);
```

The facility provides compile-time safety through automatic size verification, eliminates error-prone manual count calculations, and makes the intent explicit.

Although the underlying idea also appears applicable to other homogeneous array-like abstractions, this paper proposes only the `std::simd` facility. Possible extensions to `std::array` and `std::span` are discussed as future directions, since those types raise separate design questions.

This proposal requires array-like layout guarantees being developed for `std::simd`, and those have wider implications which are discussed in [P3983R0]. It represents a focused improvement to make element reinterpretation as natural and safe in portable C++ as it already is in platform-specific intrinsics.

§ 2. Revision History

R0 => R1

- Typos and minor rendering fixes.
- Reframed generalization in terms of homogeneous array-like types rather than contiguous containers.
- Clarified that heterogeneous product types such as `pair` and `tuple` are not natural targets for this facility.
- Strengthened discussion of the dependency on array-like layout guarantees.
- Gave more implementation freedom to return type of `bit_cast_as`.

§ 3. Motivation

§ 3.1. The SIMD Element Reinterpretation Problem

SIMD programming frequently requires reinterpreting vector data at different element granularities. Common scenarios include:

§ 3.1.1. Signal Processing - Packed Sample Conversion

```
// Receive 16 packed 8-bit samples
vec<uint8_t, 16> samples_8bit = receive_audio();

// Need to process as 8 16-bit samples
// Current approach: verbose and error-prone
auto samples_16bit = std::bit_cast<vec<uint16_t, 8>>(samples_8bit);
// Must manually specify element count          ^^^
```

§ 3.1.2. Bit Manipulation - Type Punning

```
// Get bit representation of floats for IEEE 754 operations
vec<float, 8> floats = /*...*/;
```

```
// Current: must know exact result type
auto bits = std::bit_cast<vec<uint32_t, 8>>(floats);
//          ^^^ magic number
```

§ 3.1.3. Data Packing - Bytes to Words

```
// Combine byte pairs into 16-bit values
vec<uint8_t, 32> bytes = load_packed_data();

// Current: compute count manually, specify Abi explicitly
using target_abi = /* ??? */;
auto words = std::bit_cast<vec<uint16_t, 16>>(bytes);
```

These examples all use the same primitive operation: preserve the underlying bits while reinterpreting them as a different element type, with the destination element count determined mechanically from the total size.

§ 3.2. Ininsics Already Support This Pattern

Platform intrinsics have long supported element reinterpretation without needing to specify counts:

```
// x86 intrinsics - type changes, same register
__m256i bytes_vec = _mm256_loadu_si256(/*...*/);
__m256i shorts_vec = bytes_vec; // Same bits, different interpretation

// Explicit reinterpret intrinsics
__m256 floats = _mm256_loadu_ps(/*...*/);
__m256i bits = _mm256_castps_si256(floats); // float[8] → int32[8]
```

The platform already does this naturally but `std::simd` makes it awkward.

§ 3.3. Why `std::bit_cast` Doesn't Solve This

While `std::bit_cast` handles type reinterpretation, it requires explicitly specifying the target type including element count:

```
// bit_cast requires spelling out the complete type
auto shorts = std::bit_cast<std::simd<uint16_t, 8>>(bytes);
//                                     ^^^ must specify
```

This is particularly problematic for generic code where the element count must be computed:

```
template<typename NewT, typename T, typename Abi>
auto reinterpret_elements(basic_vec<T, Abi> v) {
    constexpr size_t old_count = basic_vec<T, Abi>::size();
    constexpr size_t new_count = old_count * sizeof(T) / sizeof(NewT);
    using new_vec = resize_t<new_count, rebind_t<NewT, basic_vec<T, Abi>>>
    return std::bit_cast<new_vec>(v);
}
```

Problems with this approach:

- **Verbose:** must manually compute element count and construct ABI
- **Error-prone:** possible to get the count calculation wrong
- **Unclear:** the operation is “reinterpret elements” but the code does not say that
- **Fragile:** changes to one type parameter require recalculating everything

A natural solution is to provide a standard facility that automates this pattern, eliminating the manual computation and potential for error.

§ 3.4. What We Want

```
vec<uint8_t, 16> bytes = receive_data();

// Clear, safe, concise
auto shorts = std::bit_cast_as<uint16_t>(bytes);
// Returns vec<uint16_t, 8> – count inferred automatically
```

Benefits:

- **Automatic:** element count computed from sizes
- **Safe:** compile-time verification that sizes match
- **Clear:** intent is obvious
- **Generic:** works in templates without manual ABI computation

§ 4. Scope and Design Boundary

This proposal is not intended as a general facility for arbitrary trivially copyable types, nor for all contiguous containers. Its natural domain is narrower: homogeneous array-like types whose representation corresponds to a contiguous sequence of elements of a single type, and potentially views over such sequences.

`std::simd` is a clear instance of such a type. It has a homogeneous element type, a statically known total size, and a natural interpretation of “reinterpret these bits as elements of another type”.

By contrast, heterogeneous product types such as `std::pair` and `std::tuple` are not natural targets for this operation, even when some instantiations are trivially copyable. Those types do not model a homogeneous sequence of elements, and their semantics are not those of element-granularity reinterpretation.

This proposal therefore focuses only on `std::simd`. Possible future directions for `std::array` and `std::span` are discussed later, but are not part of the proposed wording.

§ 5. Proposed Solution

We propose adding `std::bit_cast_as<T>()` to `<simd>`:

```
namespace std {
    template<typename T, typename U, typename Abi>
    basic_vec<T, /* computed Abi */> bit_cast_as(const basic_vec<U, Abi>& v)
}
```

Effect: Returns a `simd` object with element type `T` containing the same bits as `v`, with element count automatically inferred.

Constraints:

- Sizes must match exactly: `sizeof(U) * simd<U, Abi>::size() == sizeof(T) * new_count`
- Result type must be valid: `basic_vec<T, computed_Abi>` must be well-formed

§ 5.1. Usage Examples

```
// basic element reinterpretation
vec<uint8_t, 16> bytes = /*...*/;
auto shorts = std::bit_cast_as<uint16_t>(bytes); // vec<uint16_t, 8>
auto ints    = std::bit_cast_as<uint32_t>(bytes); // vec<uint32_t, 4>
auto longs   = std::bit_cast_as<uint64_t>(bytes); // vec<uint64_t, 2>
```

```
// Float/int type punning
vec<float, 8> floats = /*...*/;
auto bits = std::bit_cast_as<uint32_t>(floats); // Access bit representation

// Manipulate bits
bits &= 0x7FFFFFFF; // Clear sign bit

// Convert back
auto abs_floats = std::bit_cast_as<float>(bits);
```

```
// Generic SIMD code
template<typename T, typename Abi>
auto as_bytes(const basic_vec<T, Abi>& v) {
    return std::bit_cast_as<std::byte>(v);
}

template<typename T, typename U, typename Abi>
auto convert_elements(const basic_vec<U, Abi>& v) {
    return std::bit_cast_as<T>(v);
}
```

```
// Compile-time safety
vec<uint8_t, 15> odd_size = /*...*/;

// Error: 15 bytes doesn't evenly divide into uint32_t
auto bad = std::bit_cast_as<uint32_t>(odd_size); // Won't compile
```


§ 5.2. Relationship to Existing Facilities

Compared to `std::bit_cast`:

- `bit_cast` requires fully specifying the target type including count
- `bit_cast_as` infers the count automatically
- `bit_cast_as` uses existing `simd` type machinery (`rebind_t`, `resize_t`) in valid ways
- `bit_cast_as` is more ergonomic for the common "reinterpret elements" use case

Compared to intrinsics:

- intrinsics already support this naturally: `_mm256_castps_si256(vec)`
- `std::simd` should provide equivalent expressiveness
- But with better type safety and generic programming support

§ 6. Design Decisions

§ 6.1. Element Count Inference

We propose that the element count should be automatically inferred from the sizes.

```
// User specifies only element type
auto result = bit_cast_as<uint16_t>(vec);

// NOT: bit_cast_as<uint16_t, 8>(vec) // Explicit count rejected
```

The rationale for automatic inference:

- safer — prevents count/size mismatches
- more concise — especially in generic code
- matches how intrinsics work (`_mm256_castps_si256` does not take a count)
- matches `std::bit_cast` philosophy (sizes determine validity)
- there is no evident use case where an explicit count adds value

§ 6.2. Size Mismatch Handling

We require exact size match with a compilation error otherwise:

```
vec<uint8_t, 15> vec;  
auto bad = bit_cast_as<uint32_t>(vec); // Error: 15 bytes != N * 4 bytes
```

The rationale for this is:

- matches `std::bit_cast` behavior
- prevents silent data loss
- prevents undefined behavior
- requires no runtime support — this is fundamentally a compile-time operation

§ 6.3. Abi Computation Strategy

The required ABI changes arising from changing the element type in terms of existing features from the draft standard, namely `rebind_t` and `resize_t`:

```
template<typename T, typename U, typename Abi>  
auto bit_cast_as(const basic_vec<U, Abi>& v) {  
    constexpr size_t old_count = simd<U, Abi>::size();  
    constexpr size_t new_count = old_count * sizeof(U) / sizeof(T);  
  
    // Step 1: Rebind to new element type  
    using new_type = rebind_t<T, basic_vec<U, Abi>>;  
  
    // Step 2: Resize to new element count  
    using new_vec = resize_t<new_count, new_type>;  
  
    return new_vec{/* bit reinterpretation */};  
}
```

Rationale:

- reuses existing type machinery from the draft standard.
- yields a well-formed result when `rebind_t`/`resize_t` are well-formed, with ABI/layout correctness discussed in the issue below.

§ 6.4. Implementation Mechanism

With the array-like layout guarantee of [P3983R0], implementations can realize this operation in a straight-forward way. In many cases this may be achievable with no or minimal overhead, similarly to intrinsic reinterpretation operations.

However, the function should not be specified in terms of `bit_cast` between the concrete source and destination `simd` types. Different valid `simd` instantiations may use different ABI-dependent layouts, alignments, or amounts of padding even when they represent the same number of element bits.

Without the array-like layout guarantees, such a `bit_cast` would not portably guarantee the intended semantics of element-granularity reinterpretation.

Platform intrinsics already support this because they make implicit array-like guarantees. This proposal brings `std::simd` to parity with intrinsics.

§ 6.5. Naming

`bit_cast_as` was chosen because:

- **Clear relationship to existing bit cast:** it immediately signals that this is a bit-level reinterpretation, not a conversion
- **Distinguishes from full-type `bit_cast`:** `bit_cast<T>(x)` selects the entire destination type, while `bit_cast_as<T>(x)` selects only the destination element type and leaves the rest to be inferred
- **Explicit about the operation:** it makes clear that this is about reinterpreting bits, not changing element values
- **Natural extension of existing vocabulary:** the `_as<T>` suffix suggests “interpret as `T`”

Another strong contender for the name was `as_elements`:

- While shorter and matching the `as_bytes` pattern from `span`, it doesn’t clearly convey that this is a bit-level reinterpretation
- `as_elements` could be confused with accessing or viewing elements, rather than reinterpreting bits
- The relationship to `bit_cast` is less obvious, making it harder for users to understand the operation’s guarantees and constraints

A few weaker alternatives were also considered:

- `simd_bit_cast<T>` - Our original name from the previous paper [P3445R0], but it is too narrow and doesn't indicate element reinterpretation. `std::simd` also dropped the `simd_` prefix in favour of a namespace, and applying that change here results in plain `bit_cast` as the name, which is clearly a bad name.
- `reshape_as<T>` - unclear whether it changes dimensions or element type
- `reinterpret_as<T>` - matches `reinterpret_cast` but less specific

§ 7. Implementation Experience

In Intel's implementation of `std::simd` the original element bit casting function called `simd_bit_cast` was added very early on because it is so widely used.

The implementation of Intel's `std::simd` library itself uses the element bit casting to make it easier to interface to compiler intrinsics. Intrinsics often require particular data types to be used to achieve certain effects, and the bit-cast allows the underlying bits to be quickly and easily reinterpreted.

Intel uses `std::simd` in a number of internal software projects, and some of those (particularly wireless or packet-processing) need to be able to easily reinterpret the underlying bits in different ways. Some of those software projects were originally written in plain intrinsics and then rewritten to use `std::simd`. In those projects intrinsics like `_mm256_castps_si256` were used, and `bit_cast_as` provides the natural equivalent.

§ 7.1. Conceptual Implementation

```
template<typename T, typename U, typename Abi>
auto bit_cast_as(const simd<U, Abi>& v) noexcept {
    constexpr size_t old_bytes = sizeof(U) * simd<U, Abi>::size();
    constexpr size_t new_count = old_bytes / sizeof(T);
    static_assert(old_bytes % sizeof(T) == 0, "Size mismatch");

    using new_abi = simd_abi::resize_t<new_count, simd_abi::rebind_t<T, Abi>::size_t>;

    return std::bit_cast<simd<T, new_abi>>(v);
}
```

§ 8. Future Directions

The operation proposed here appears to generalize beyond `std::simd`, but not to arbitrary containers or tuple-like types. The natural generalization axis is homogeneous array-like types and, potentially, views over homogeneous contiguous storage.

§ 8.1. Possible Extension to `std::array`

The same operation applies naturally to `std::array`, which is the fixed-size homogeneous value-type analogue of `simd`.

```
std::array<uint8_t, 16> bytes = /*...*/;
auto shorts = std::bit_cast_as<uint16_t>(bytes); // array<uint16_t, 8>
```

For `array`, `bit_cast_as` would return a new value, just as it does for `simd`. The destination extent is determined mechanically from the source extent and the element sizes.

A possible design would be:

```
template<class T, class U, size_t N>
    requires (sizeof(U) * N % sizeof(T) == 0) &&
               is_trivially_copyable_v<T> &&
               is_trivially_copyable_v<U>
constexpr array<T, sizeof(U) * N / sizeof(T)>
bit_cast_as(const array<U, N>& a) noexcept {
    return std::bit_cast<array<T, sizeof(U) * N / sizeof(T)>>(a);
}
```

This is a particularly clean extension because `array` already has fixed extent and homogeneous element structure.

§ 8.2. Possible Extension to `std::span`

A corresponding operation also appears meaningful for `std::span`, but in that case the result would be a view, not a new value.

```
std::span<const uint32_t, 8> words = /*...*/;
```

```
auto bytes = std::bit_cast_as<const std::byte>(words); // span<const byte
```

This is conceptually similar to `as_bytes`, but generalized to arbitrary destination element types rather than only `std::byte`.

However, unlike the `simd` and `array` cases, a `span`-based facility raises additional questions:

- alignment requirements for the destination element type
- cv-qualification propagation
- interaction with aliasing rules
- interaction with existing `as_bytes` and `as_writable_bytes`
- handling of dynamic extent

These questions are separable from the `simd` motivation of this paper. For that reason, `span` is best viewed as a possible future extension rather than part of the present proposal.

§ 8.3. Non-Targets

Heterogeneous product types such as `std::pair` and `std::tuple` are not natural targets for this facility, even when some instantiations are trivially copyable. They do not model a homogeneous sequence of elements, and their semantics are not those of element-granularity reinterpretation.

Similarly, the intended generalization is not to all contiguous containers. Dynamic owning containers such as `std::vector` are adequately served by first forming a `span`, should a suitable `span`-based facility ever be standardized.

§ 9. Design Alternatives Considered

§ 9.1. Alternative: Make this a member function

```
auto shorts = bytes.bit_cast_as<uint16_t>();
```

Rejected because:

- inconsistent with free-function vocabulary such as `as_bytes`
- less flexible for future extension to other types

- a free function better communicates that this is a generic library operation

§ 9.2. Alternative: Use explicit count parameter

```
auto result = bit_cast_as<uint16_t, 8>(bytes);
```

Rejected because:

- redundant — the count is determined by the sizes
- error-prone — users could specify an inconsistent count
- less convenient in generic code
- inconsistent with the intended design (i.e., the user should not have to specify anything that can be inferred)

§ 9.3. Alternative: Generalize immediately to `array` and `span`

This paper deliberately does not propose wording for `array` or `span`.

`array` appears to be a natural fit, but broadening the wording would increase the scope of the paper beyond the motivating `simd` facility.

`span` is more complicated still, because its design space involves view semantics, alignment, aliasing, and interaction with existing byte-view facilities.

A narrower `simd`-only paper is therefore more likely to receive focused review on its core merits.

§ 10. Wording

Tentative wording for the initial proposal.

§ 10.1. Header `<simd>` synopsis additions

```
namespace std {  
    template<class T, class U, class Abi>  
        simd<T, /* see below */>
```

```
    bit_cast_as(const simd<U, Abi>& v) noexcept;
}
```

§ 10.2. `bit_cast_as` for `simd` [`simd.bit_cast_as`]

```
template<typename T, typename U, typename Abi>
simd<T, /* see below */> bit_cast_as(const simd<U, Abi>& v) noexcept;
```

Constraints:

- `sizeof(U) * simd<U, Abi>::size() % sizeof(T) == 0`
- `is_trivially_copyable_v<T>` is true
- `is_trivially_copyable_v<U>` is true
- Let `new_count` be `sizeof(U) * simd<U, Abi>::size() / sizeof(T)`.
- There exists an ABI type `NewAbi` such that `simd<T, NewAbi>` is a valid, complete type and `simd<T, NewAbi>::size() == new_count`.

Returns: A value of type `simd<T, NewAbi>` for some ABI type `NewAbi` satisfying the constraints above.

Postconditions: Let `S` be the sequence of `sizeof(U) * simd<U, Abi>::size()` bytes comprising the elements of `v` in element order. Let `R` be the sequence of `sizeof(T) * simd<T, NewAbi>::size()` bytes comprising the elements of the returned value in element order. `S` and `R` are equal.

Remarks:

- The behavior described in this subclause requires `simd` to have array-like layout as specified in [P3983R0].

§ 10.3. Feature Test Macro

Add to `<version>`:

```
#define __cpp_lib_simd_bit_cast_as 202601L // also in <simd>
```


§ 11. Acknowledgements

Thanks to Matthias Kretz for review and feedback on the specification, particularly around representation constraints and return-type over-specification.